



APIs alternativas





GraphQL

GraphQL



GraphQL

**GraphQL es
un lenguaje de consulta y manipulación de datos para
APIs,
y un entorno de ejecución para realizar consultas con
datos existentes**

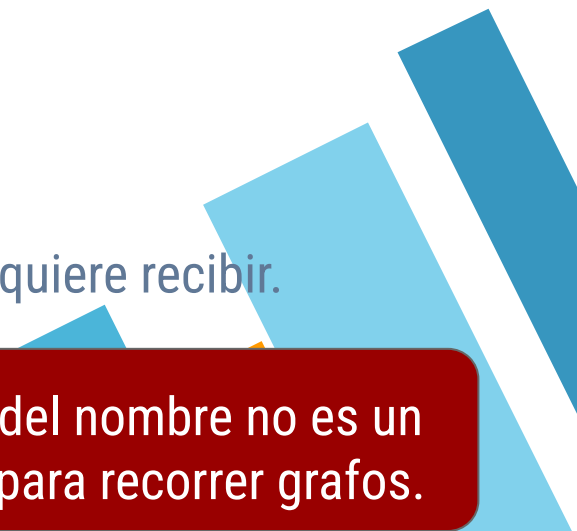




GraphQL

GraphQL permite definir APIs Web

- Con un esquema tipado
- Agnóstico del transporte y la fuente de datos.
- Explorable y descubrible.
- Con operaciones de:
 - búsqueda (queries)
 - modificación (mutation)
 - subscripción (subscription).
- Con la posibilidad de que el cliente seleccione lo que quiere recibir.



Más allá del nombre no es un lenguaje para recorrer grafos.

GraphQL - Ventajas

Como ventajas no evidentes al compararlo con REST se destacan:

- Se pueden reducir los llamados al servidor para obtener los mismos datos
- Se pueden solicitar sólo los datos que se precisan
- Esos datos pueden provenir de múltiples fuentes.

```
"recentPosts": [  
  {  
    "id": "Post96",  
    "title": "Post 9:6",  
    "category": "Post 6 + by author 9",  
    "text": "Post category",  
    "author": {  
      "id": "Author9",  
      "name": "Author 9"  
    }  
  }  
]
```



GraphQL - Esquema

En GraphQL la API se debe definir mediante un [esquema](#).

Este esquema va a ser fuertemente tipado y va a definir los objetos que definen argumentos y respuestas otorgándole un estilo similar a una API RPC.

El esquema se puede publicar en el servicio y a partir del mismo el cliente puede inferir las operaciones.

El discovery y la inferencia son fáciles porque el servicio se publica en una url y todas las operaciones se ejecutan en ese endpoint con un request POST variando el body de acuerdo al parámetro.



GraphQL - Tipos

El esquema en GraphQL consta de tipos

```
type Author {  
  id: ID!  
  name: String!  
  posts: [Post]!  
  thumbnail: String  
}
```

```
type Post {  
  author: Author!  
  category: String  
  id: ID!  
  text: String!  
  title: String!  
}
```

donde:

- Cada campo debe tener un nombre y un tipo
- Los tipos pueden ser **String, Int, Float, Boolean, and ID**
- **!** indica que el campo es obligatorio (non null)
- **[]** indica que se devuelve un array de valores.

GraphQL - Operaciones

Para las operaciones existen tipos especiales para las diferentes operaciones.

Entonces se puede definir un type Query (obligatorio), un type Mutation y una Subscription. Cada uno de ellos tienen métodos en vez de campos.

Opcionalmente se los puede wrappear en un schema

```
schema {  
  query: Query  
  mutation: Mutation  
}  
  
type Query {  
  ...  
}  
  
type Mutation {  
  ...  
}
```

GraphQL - Query

El tipo Query tiene todos los métodos que realizan consultas a los datos. Cada método tiene nombre, parámetros tipados y tipos de respuesta.

```
type Query {  
  recentPosts(count: Int, offset: Int): [Post]!  
}
```

GraphQL - Hacer una Query

A la hora de realizar un llamado a una query el body se prefija con la keyword query opcionalmente se define parámetros para la llamada y se pasa un objeto con el nombre del objeto a llamar los parámetros y campos que se espera en la respuesta

```
query myRecentPosts($count: Int, $offset: Int) {  
  recentPosts(count: $count, offset: $offset) {  
    id  
    title  
    text  
    category,  
    author {  
      id  
      posts {  
        id  
      }  
    }  
  }  
}
```

En caso de parametrizar el llamado los argumentos se pasan en un objeto aparte

```
{  
  "count": 2,  
  "offset": 2  
}
```

Ejercicio 1

- Bajar el proyecto graphql-intro
- Analizar el contenido
- Realizar la query de los posts pidiendo el id y el titulo.



GraphQL - Query Resolver

Para poder responder las consultas del cliente hay que implementar algún elemento que pueda responder.

En GraphQL esos elementos se llaman resolvers hay diversos tipos de resolvers de acuerdo a lo que resuelven (Query, Mutation, Subscription, Field) y eso es lo que permite tener granularidad a la hora de completar los campos pedidos con datos que provienen de diversas fuentes.





GraphQL - Query Resolver - Código

La generación de objetos de modelo y resolvers depende mucho del lenguaje y la librería que se esté utilizando.

Existen los que son solo Middleware, los que generan código a partir del esquema, los que generan el esquema a partir del código y anotaciones, etc.



GraphQL - Query Resolver - Java (Spring)

Existen diversas implementaciones de GraphQL en Java, particularmente en estos ejemplos usamos la de Spring que wrappea la oficial de GraphQL.

Entonces para definir un resolver definimos un **Controller** que contiene a un método anotado como **QueryMapping**.

```
@Controller
public class PostController {

    @QueryMapping
    public List<Post> recentPosts(@Argument int count,
    @Argument int offset) {
        return ...;
    }
}
```

Ejercicio 2

- Implementar el query resolver.
- Probar la query planteada en el ejercicio anterior.
- Agregar pedir el autor

GraphQL - Field Resolver

En Java Spring para resolver un campo utilizamos la annotation **SchemaMapping** para algún método que esté dentro de algún **Controller**.

```
@SchemaMapping(typeName = "Post", field = "author")
public Author getAuthor(Post post) {
    return ...;
}
```

GraphQL - Field Resolver

Los Field Resolvers permiten resolver de que manera se obtiene el contenido de algún campo dentro de un tipo de respuesta de una query.

Se lo puede hacer tan granular como para decir “para este campo de este tipo” se resuelve con con este resolver.

Esto es lo que permite devolver una respuesta que se compone con información de diversas fuentes

```
@SchemaMapping(typeName = "Post", field = "author")
public Author getAuthor(Post post) {
    return ...;
}
```

Ejercicio 3

- Implementar el field resolver para el autor del post.



GraphQL - Batch Loaders

Batch Loaders es un concepto que se crea para mitigar el riesgo de caer en el problema de $n + 1$ queries.

Este problema se da cuando yo solicito un listado de elementos que requiere para resolver parte de su contenido hacer una query más por cada elemento del sistema.

En nuestro caso por cada post necesitamos “resolver” el autor, esto requiere que se llame al field resolver 1 vez por cada post.

Obviamente tener una resolución con comportamiento lineal no es óptimo





GraphQL - Batch Loaders

Al registrarle un Batch Loader a un Field lo que hace GraphQL es acumular los pedidos a ese campo (que generalmente es por id) y en vez de hacer n llamados con 1 elemento, hace 1 llamado con los n elementos en un array.

De esta manera quien implemente puede resolver la respuesta de una manera más eficiente



GraphQL - Batch Loaders

En java spring la manera más simple de implementar un BatchLoader es anotar un método como **BatchMapping** e implementarlo viendo que el parámetro es una lista de los elementos “padre” y la respuesta es un Mono<Map<Padre, Hijo>>

```
@BatchMapping
```

```
public Mono<Map<Post, Author>> author(List<Post> posts) {  
    Map<Post, Author> authors = new HashMap<>();  
  
    return Mono.just(authors);  
}
```

Ejercicio 4

- Implementar el batch loading para los autores de los posts/
- Probar que se cumple el llamado optimizado.



GraphQL - Mutations

Una mutación se define de manera similar a una query.

```
type Mutation {  
  createPost(authorId: String!, category: String, text: String!,  
title: String!): Post!  
}
```



GraphQL - Mutations

En la consulta, también similar a la query el body pasa los argumentos de creación y define los campos de la respuesta que espera

```
mutation createPost(  
  $title: String!  
  $text: String!  
  $category: String  
  $authorId: String!  
) {  
  createPost(  
    title: $title  
    text: $text  
    category: $category  
    authorId: $authorId  
  ) {  
    id  
    title  
    text  
    category  
  }  
}
```

```
{  
  "title": "Make your own budget",  
  "text": "Do it at the beggining of the month",  
  "category": "self economy ",  
  "authorId": "Author1"  
}
```

GraphQL - Mutation Resolver

Para implementar el resolver para la mutación anotar el método con **@MutationMapping**

```
@MutationMapping
public Post createPost(@Argument String title, @Argument String text,
                      @Argument String category, @Argument String authorId) {

    Post post = new Post(
        UUID.randomUUID().toString(),
        title,
        text,
        category,
        authorId);

    postDao.savePost(post);

    return post;
}
```

Ejercicio 5

- Implementar la mutación para los posts.



APIs de Eventos





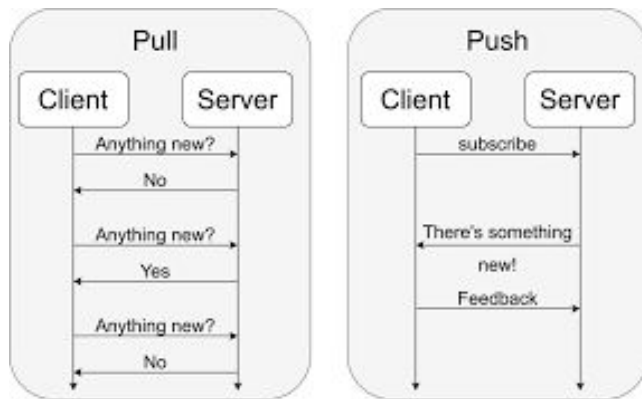
Event Driven

Las Apis Event first o Event driven donde la comunicación de elementos relevantes al Cliente se inicia al momento de que se produce el Evento.



Web Hooks

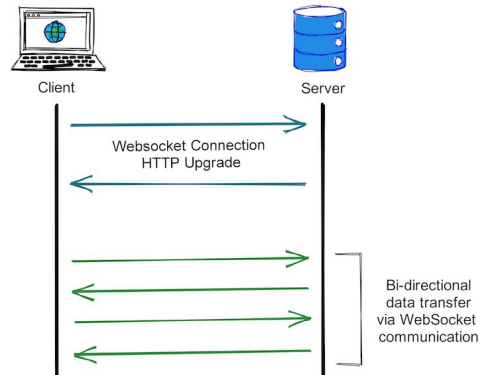
Un webhook es un "callback" HTTP automatizado que se dispara cuando pasa algo específico en un sistema origen. El sistema manda la información a la URL que el cliente configura apenas ocurre el evento. Ejemplos de uso: alertas de pago, notificaciones de Git o automatizaciones (por ejemplo CI/CD)



Web Sockets

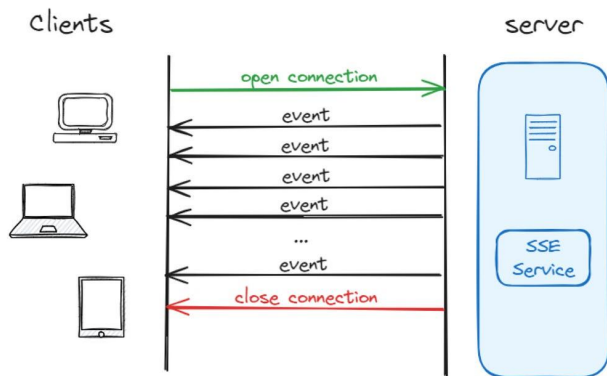
Web Sockets es una estrategia comunicacional donde el cliente abre una conexión "permanente" con el servidor y por la misma se realiza una comunicación full duplex entre ambos. Se puede utilizar en aplicaciones de chat, juegos multijugador online o herramientas colaborativas tipo Figma.

Una implementación de Subscriptions pro GraphQL usa web sockets



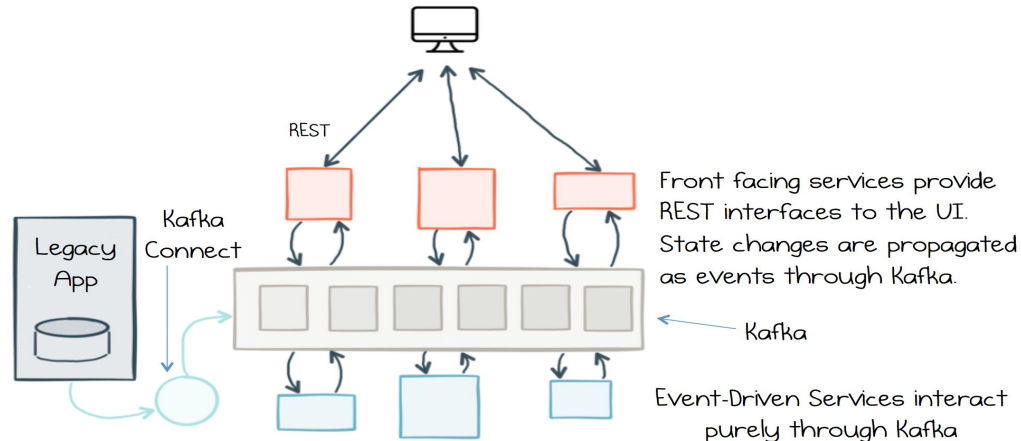
Server-Sent Events (SSE)

Server-Sent Events (SSE) es una tecnología donde el cliente abre una conexión permanente con el servidor y este le va mandando actualizaciones en tiempo real. La comunicación es unidireccional (va solo del servidor al cliente). Es liviano, se reconecta solo si se cae. Ejemplos de uso: a tableros con datos en vivo, cotizaciones de acciones o feeds de noticias que se actualizan solas.



PubSub via Mensajería

Usa un intermediario (un broker) para pasar datos de forma asincrónica entre distintos servicios. Un componente (productor) escribe el mensaje a la cola y se despreocupa; el mensaje queda guardado ahí de forma segura hasta que otro componente (consumidor) lo procesa. Esto desacopla totalmente los sistemas: si un servicio se cae, la data no se pierde. Ejemplos: Kafka, RabbitMQ.





Takeaways

- » Existen muchas maneras de definir APIs
 - » Las dos corrientes más grandes son Request/Response y Eventos
 - » Dentro de las mismas hay diversas tecnologías, modos y frameworks que puedo elegir de acuerdo a las necesidades particulares de mis sistema.
 - » Dentro de cada una de ellas muchos de los elementos discutidos cuando vimos gRPC aplican a la hora de pensar/entender la construcción de los mismos.
- 



CREDITS

Content of the slides:

- » POD - ITBA

Images:

- » POD - ITBA
- » Or obtained from: commons.wikimedia.org

Slides theme credit:

Special thanks to all the people who made and released these awesome resources for free:

- » Presentation template by [SlidesCarnival](https://slidescarnival.com)
 - » Photographs by Unsplash
- 